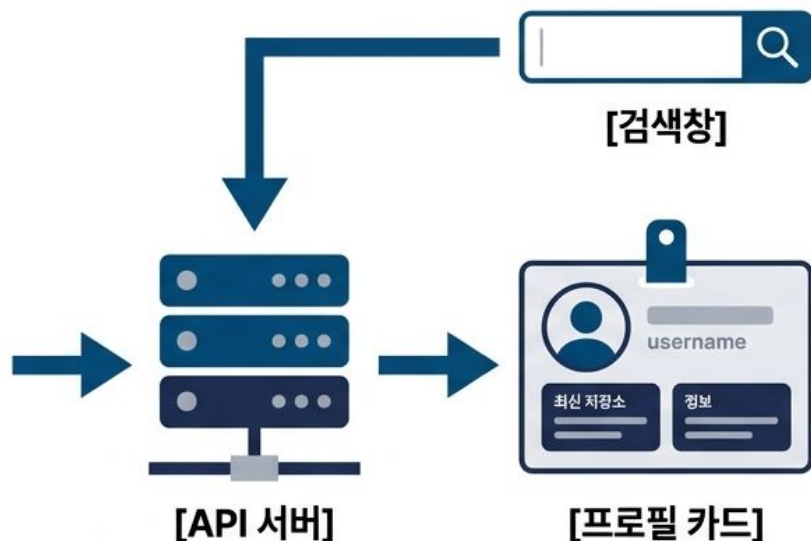


자바스크립트를 이용한 GitHub Finder 앱 만들기

과제 3. GitHub Finder 앱 만들기

[Mission Objective] 단순 화면 모방이 아닌, API 데이터 기반의 UI 구성

- **핵심 목표:** 단순 화면 렌더링이 아닌, 실제 API 데이터 연동 및 시각화
- **주요 기능 1:** GitHub 사용자명 입력 및 실시간 검색
- **주요 기능 2:** GitHub API 연동을 통한 사용자 데이터 수신
- **주요 기능 3:** 사용자 프로필 및 최신 저장소 목록 화면 출력
- **주요 기능 4:** 로딩, 빈 결과, 에러 상태의 안정적 예외 처리



오늘의 운영 방식

[Timeline] 2시간 세션 타임라인 및 단계별 목표

전반부 (1시간 실습)



후반부 (1시간 피드백)

■ 운영 개요:

총 2시간 집중 세션
(1차 핵심 구현 →
라이브 피드백)

■ 전반부 (1시간 실습):

과제 요구사항 분석 및
실시간 핵심 흐름 구현

■ 후반부 (1시간 피드백):

공통 라이브 피드백 및
코드 점검

■ 마무리 (제출 안내):

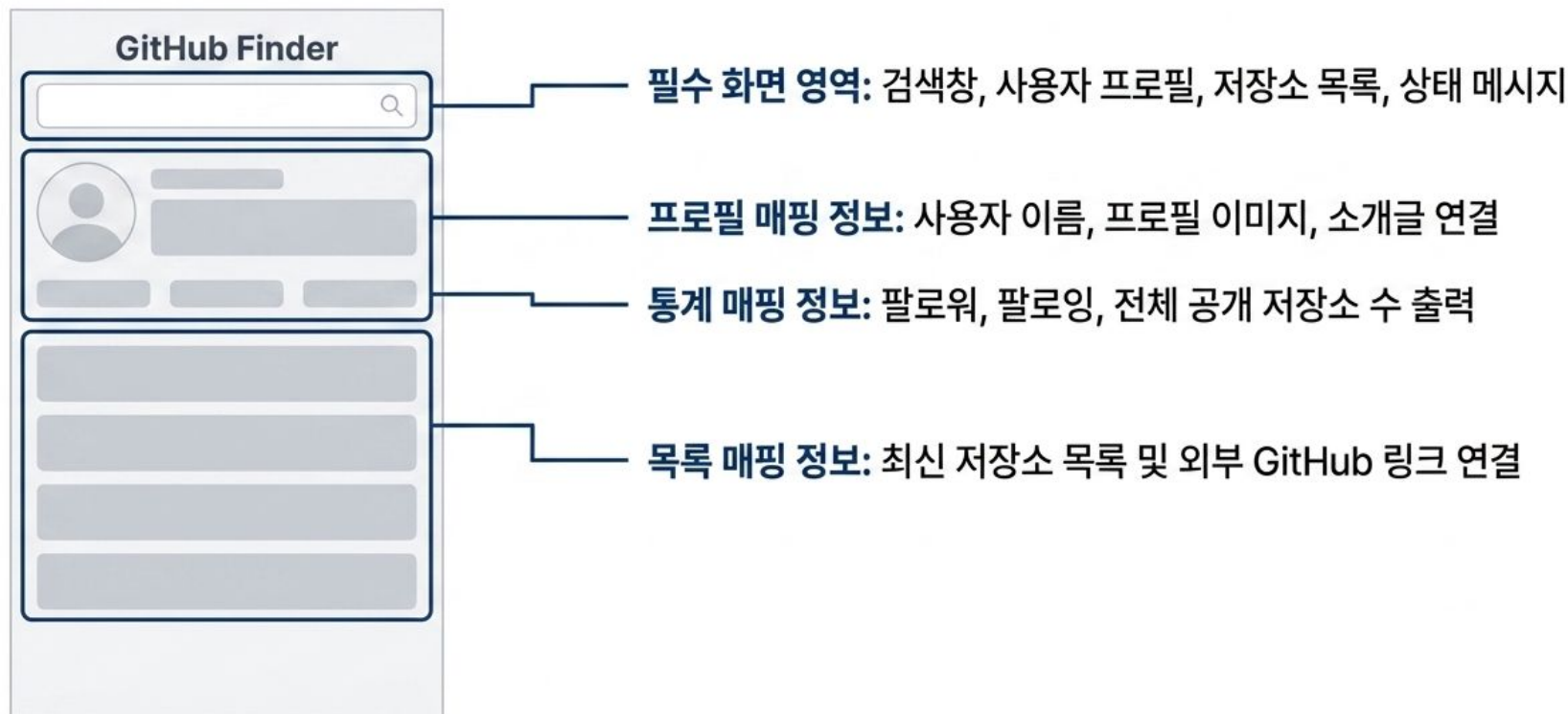
최종 보완 방향 설정 및
README 작성 가이드

■ 핵심 지향점:

완벽한 디자인 완성보다
데이터 흐름의 연결 우선

최종 결과물의 방향

[Architecture] 시각적 디자인보다 구조와 데이터 연결 우선



필수 및 선택 구현 기능

[Prioritization] 구현 우선순위 명확화 (Must-have vs Nice-to-have)

- ☑ [필수] 데이터 요청: 사용자명 검색 및 GitHub API 데이터 요청
- ☑ [필수] 화면 출력: 응답받은 프로필 및 저장소 목록 데이터 렌더링
- ☑ [필수] 예외 처리: 빈 입력값 및 '사용자 없음(404)' 안내 메시지 출력

- ☑ [선택] UX 개선: 로딩 메시지 표시 및 화면 크기 반응형(Responsive) 적용
- ☑ [선택] 문서화: AI 활용 과정 및 트러블슈팅 README 문서화

구현 흐름: 입력에서 출력까지

[Data Pipeline] 데이터가 처리되고 화면에 그려지는 4단계 과정



API 접근 및 데이터 분석

[Research] API 명세서 탐색 및 AI 활용 데이터 추출

URL 구조 분석: 목적에 맞는 API 엔드포인트 및 요청 형태 파악

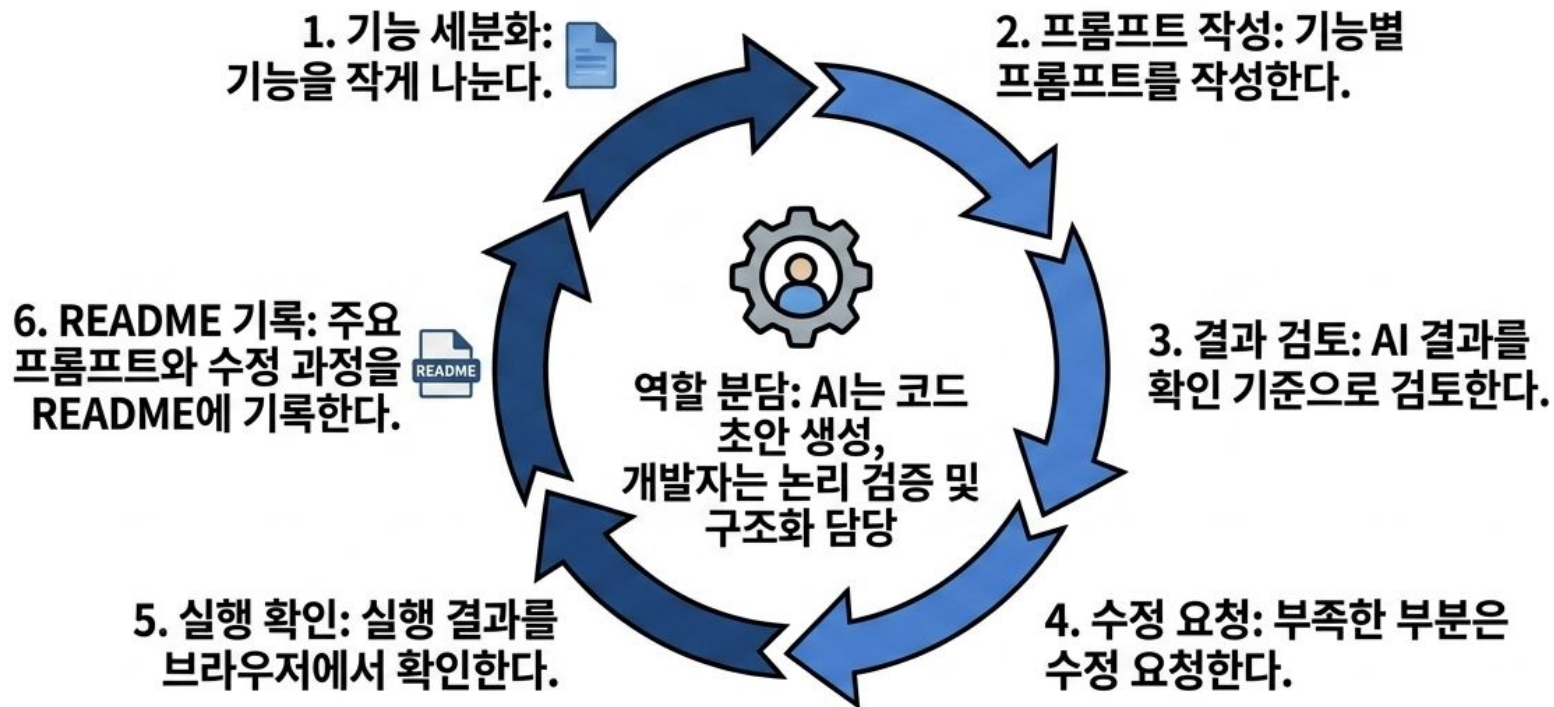
응답 데이터 분석: JSON 응답 구조 내 필수 필드 및 데이터 타입 확인

실패 상황 예측: 404 Not Found 등 발생 가능한 에러 코드 사전 파악

AI 프롬프트 예시: GitHub REST API에서 사용자 정보 및 저장소 목록을 가져오는 방법 정리 요청

바이브 코딩 진행 방식

[Workflow] AI와의 협업을 통한 코딩 프로세스 구조화



실시간 수행 체크리스트 (1차 목표)

[Action Plan] 1시간 내 완수해야 할 최소 기능 기준점



- ☐ 초기 설정: 프로젝트 파일 구조 및 HTML DOM 뼈대 생성
- ☐ 데이터 연결: 검색창 입력값 읽기 및 API 비동기 요청(Fetch)
- ☐ 데이터 출력: 수신된 사용자 정보 및 저장소 목록 화면 렌더링
- ☐ 예외 처리: 잘못된 사용자명 입력 시 기본 오류(404) 처리 로직 적용

실시간 과제 수행
PM 07:20 - 08:20

최종 제출물 안내

[Deliverables] 결과물 제출 시 필수 포함 항목



시각적 증명: 정상 동작하는 최종 실행
화면 캡처 이미지 첨부

코드 제출: 완성된 소스 코드가 포함된
구글 드라이브 링크

중요 (Point)



[필수] AI 활용 기록: README 내
프롬프트 활용 내역 3개 이상 작성

[필수] 트러블슈팅: 개발 중 발생한
오류 및 해결 과정 1개 이상 기록

README 작성 기준 및 예시

[Documentation] 훌륭한 리드미 문서의 구성 요소 가이드

TOC

- 필수 포함 항목: 과제 개요, 주요 기능, 사용 기술 스택, 구현 흐름도
- AI 프롬프트 기록: 문제 해결을 위해 실제 사용한 질문과 AI의 답변 요약
- 트러블슈팅 (Troubleshooting): 직면한 오류 현상, 원인 분석, 최종 해결 코드
- 학습 회고: 과제를 수행하며 새롭게 배운 개념 및 보완할 점 작성

개발 기록

AI 활용 과정

프롬프트: GitHub API 요청을 비동기로 처리하는 방법 제안 요청

트러블슈팅

발생 문제: 404 에러 발생 시 UI 깨짐 현상

자주 발생하는 실수 점검

[Debugging] 오류 발생 시 스스로 점검할 수 있는 체크 포인트

- ⚠ **선택자 오류:** HTML id 속성과 JavaScript DOM 선택자 철자 불일치
- ⚠ **비동기 처리 누락:** await 키워드 누락으로 인한 Promise 객체 반환 오류
- ⚠ **데이터 구조 혼동:** 배열(Array)과 객체(Object)의 접근 방식 혼용
- ⚠ **상태 처리 미흡:** 404 에러 미처리 및 화면에 null/undefined 노출
- ⚠ **디버깅 기본기 부족:** 브라우저 개발자 도구(Console) 오류 메시지 미확인



라이브 피드백 점검 기준

[Self-Audit] 피드백 시간에 스스로 코드를 보정하는 기준점



통신 점검: API 요청 URL의
파라미터 정확성 및 JSON
응답 정상 수신 확인



렌더링 점검: 데이터 매핑
정확도 및 저장소 목록
데이터 반복 출력 여부



안정성 점검: 빈 입력값 제출
방지 및 존재하지 않는
사용자 예외 처리



기록 점검: Console 창
오류(Red log) 제로화 및 AI
활용 기록 유무



최종 제출 전 보완 방향

[Final Polish] 세션 종료 후 최종 제출까지의 완성도 향상 가이드



접근 권한 확인: 과제/개인프로젝트 드라이브 이용



UI 가시성 개선: 텍스트 오버플로우 방지
및 모바일 화면 깨짐 현상 수정



기능 무결성 검증: 검색 이벤트 정상
동작 및 에러 화면 노출 재확인

[보완 (개인)]



문서화 완비: 요구된 README 마크다운
양식 준수 및 캡처 이미지 첨부

[최종 제출 (마감)]

Office Hour /
질문 창구

[1차 구현 (오늘)]

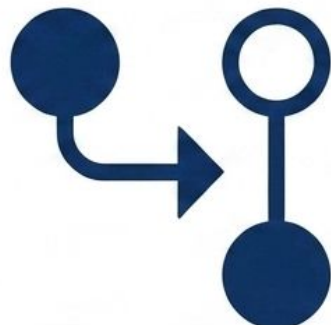
다음 특강 및 과제로 이어가기

[Synthesis] 외부 데이터 흐름 제어에서 내부 데이터 구조화로의 확장



과제 3 (현재): 외부 API 데이터의
비동기 호출, 응답 필터링 및 화면 출력

데이터 구조화 역량



특강 4 (다음): SDD 기반
내부 상태 변화 모델링 및 계산 규칙 수립

개념적 연결: 응답 데이터 읽기(API) → 모델 구조 정의(Spreadsheet)
핵심 역량: 데이터를 깊이 있게 이해하고 논리적으로 구조화하는 힘 배양